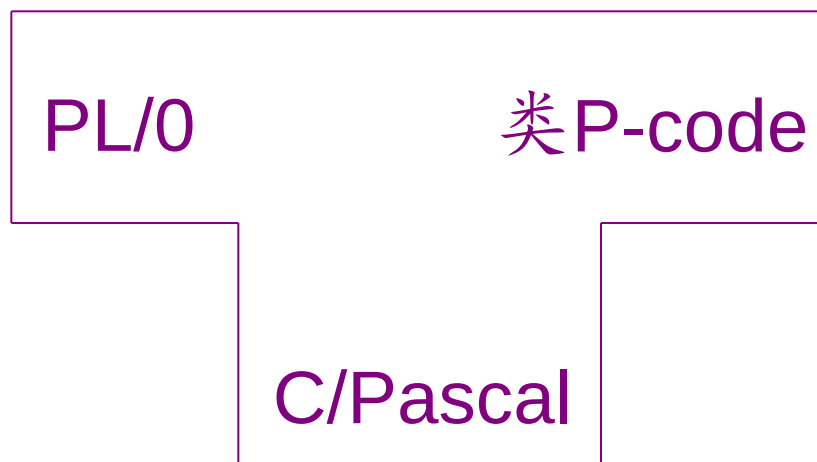


✧ PL/0 编译程序导读

- ✧ PL/0 编译程序总体结构
- ✧ PL/0 语言简介
- ✧ 类 P-code 虚拟机
- ✧ PL/0 编译程序的词法分析
- ✧ PL/0 编译程序的语法分析
- ✧ PL/0 编译程序的语义分析
- ✧ PL/0 编译程序的错误处理
- ✧ PL/0 编译程序的类 P-code 代码生成

✧ T-型图



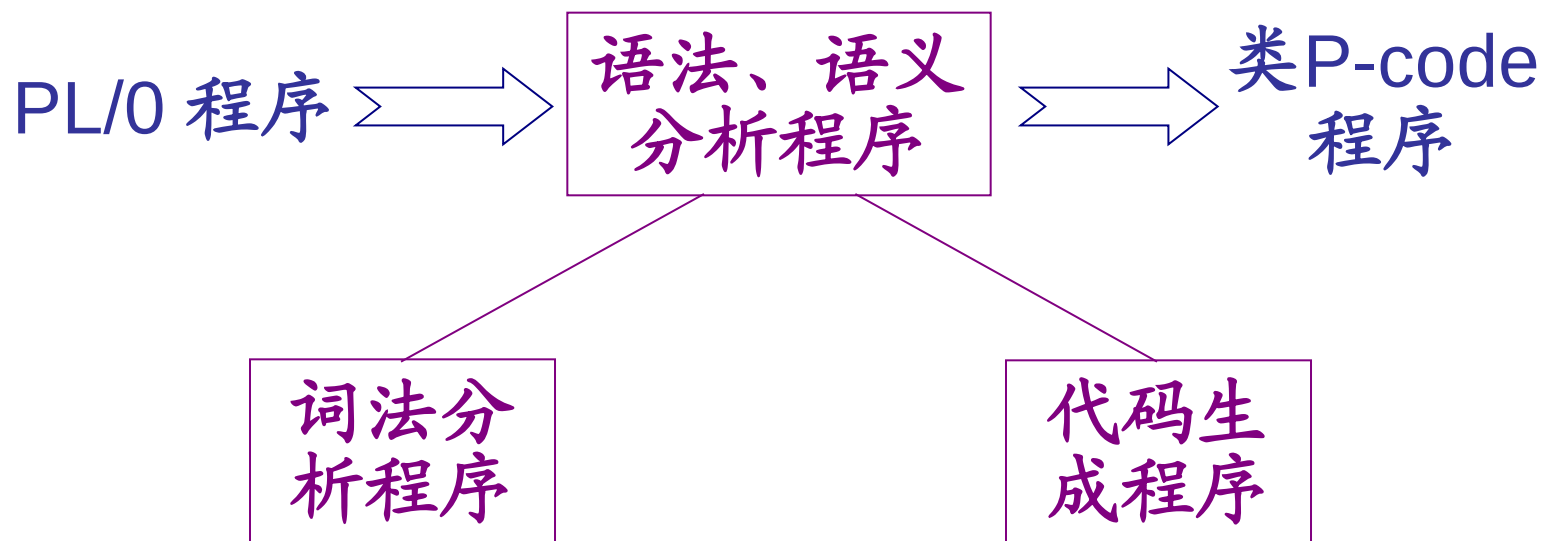
PL/0 编译程序总体结构



清华大学

《编译原理》

- ◇ PL/0 编译程序的组织：一个以语法、语义分析程序为中心的单遍编译程序



- ✧ PL/0 语言为一种简化的类Pascal 语言
- ✧ PL/0 程序示例
- ✧ PL/0 语言的语法描述图
- ✧ PL/0 语言的EBNF表示
- ✧ PL/0 语言的语义规则

PL/0 程序示例

```
CONST A=10;  
VAR B,C;  
PROCEDURE P;
```

```
  VAR D;
```

```
  PROCEDURE Q;
```

```
    VAR X;
```

```
    BEGIN
```

```
      READ(X);
```

```
      D:=X;
```

```
      WHILE X#0
```

```
      DO CALL P;
```

```
    END;
```

```
  BEGIN
```

```
    WRITE(D);
```

```
    CALL Q;
```

```
  END
```

```
BEGIN
```

```
  CALL P;
```

```
END.
```

/*主程序常量说明部分*/

/*主程序变量说明部分*/

/*主程序过程说明部分*/

/*过程P的局部变量说明部分*/

/*过程P的局部过程说明部分*/

/*过程Q的局部变量说明部分*/

Q 的过程体

P 的过程体

主程序的过程体

✧ 计算最大公约数

```
var m, n, r, q;
```

```
{ 计算m和n的最大公约数 }
```

```
procedure gcd;
```

```
begin
```

```
  while r#0 do
```

```
    begin
```

```
      q := m / n;
```

```
      r := m - q * n;
```

```
      m := n;
```

```
      n := r;
```

```
    end
```

```
end;
```

```
begin
```

```
  read(m);
```

```
  read(n);
```

```
{ 为了方便, 规定m >= n }
```

```
if m < n then
```

```
  begin
```

```
    r := m;
```

```
    m := n;
```

```
    n := r;
```

```
  end;
```

```
begin
```

```
  r:=1;
```

```
  call gcd;
```

```
  write(m);
```

```
end;
```

```
end.
```

✧ 计算

sum = 1! + 2! + ... + n!,
(n从控制台读入)

var n, m, fact, sum;

{ 递归计算 fact = m! }

procedure factorial;

begin

if m > 0 then

begin

fact := fact * m;

m := m - 1;

call factorial;

end;

end;

begin

{读入n}

read(n);

sum := 0;

while n > 0 do

begin

m := n;

fact := 1;

call factorial;

sum := sum + fact;

n := n - 1;

end;

{输出n!}

write(sum);

end.

PL/0 语言的语法描述图



清华大学

《编译原理》

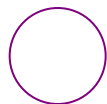
- ✧ 每个语法单位对应一个语法描述图
- ✧ 一个入口和一个出口的有向图
- ✧ 从入口可到达任何节点
- ✧ 每个节点都可以到达出口
- ✧ 从入口到出口的路径表示该语法单位的一种合法中间形式（短语）
- ✧ 有两种类型的节点



内的文字表示所用到的其他语法单位

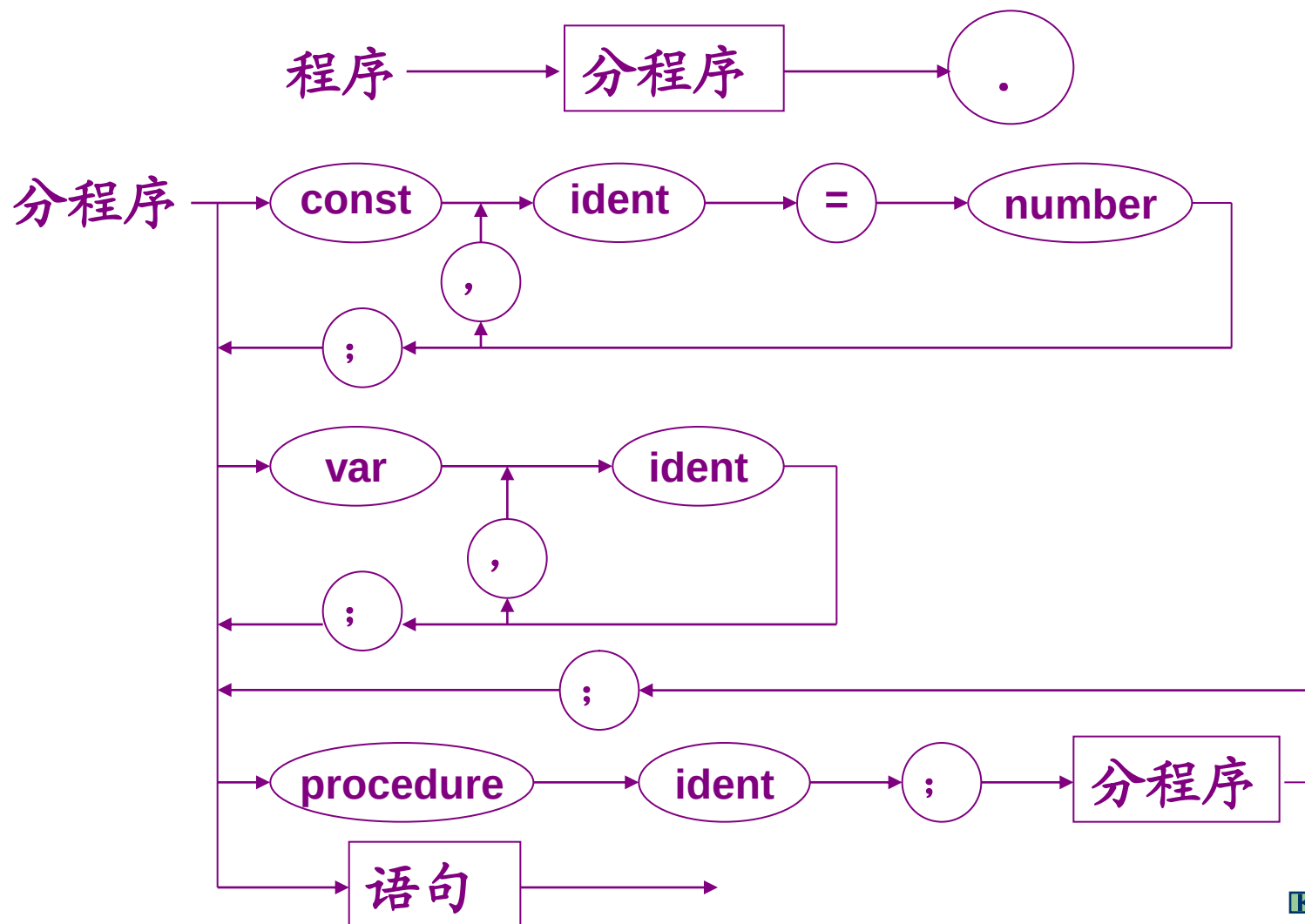


或



内的文字表示单词符号

◇ 例：程序和分程序语法单位的语法描述图



◇ EBNF 的元符号

- ‘< >’ 是用左右尖括号括起来的中文字表示语法构造成分，或称语法单位，为非终结符。
- ‘::=’ 该符号的左部由右部定义，可读作 ‘定义为’
- ‘|’ 表示 ‘或’，即左部可由多个右部定义
- ‘{ }’ 表示花括号内的语法成分可以重复；在不加上下界时可重复0到任意次数，有上下界时为可重复次数的限制
- ‘[]’ 表示方括号内的成分为任选项
- ‘()’ 表示圆括号内的成分优先

◇ 例：PL/0 语言的 EBNF 表示

$\langle \text{程序} \rangle ::= \langle \text{分程序} \rangle .$

$\langle \text{分程序} \rangle ::= [\langle \text{常量说明部分} \rangle] [\langle \text{变量说明部分} \rangle]$
 $[\langle \text{过程说明部分} \rangle] \langle \text{语句} \rangle$

$\langle \text{常量说明部分} \rangle ::= \text{CONST } \langle \text{常量定义} \rangle \{ , \langle \text{常量定义} \rangle \} ;$

$\langle \text{常量定义} \rangle ::= \langle \text{标识符} \rangle = \langle \text{无符号整数} \rangle$

$\langle \text{无符号整数} \rangle ::= \langle \text{数字} \rangle \{ \langle \text{数字} \rangle \}$

$\langle \text{变量说明部分} \rangle ::= \text{VAR } \langle \text{标识符} \rangle \{ , \langle \text{标识符} \rangle \} ;$

$\langle \text{标识符} \rangle ::= \langle \text{字母} \rangle \{ \langle \text{字母} \rangle | \langle \text{数字} \rangle \}$

$\langle \text{过程说明部分} \rangle ::= \langle \text{过程首部} \rangle \langle \text{分程序} \rangle \{ ; \langle \text{过程说明部分} \rangle \} ;$

$\langle \text{过程首部} \rangle ::= \text{PROCEDURE } \langle \text{标识符} \rangle ;$

◇ 例：PL/0 语言的 EBNF 表示

$\langle \text{语句} \rangle ::= \langle \text{赋值语句} \rangle \mid \langle \text{条件语句} \rangle \mid \langle \text{当型循环语句} \rangle \mid$
 $\langle \text{过程调用语句} \rangle \mid \langle \text{读语句} \rangle \mid \langle \text{写语句} \rangle \mid \langle \text{复合语句} \rangle \mid \langle \text{空} \rangle$

$\langle \text{赋值语句} \rangle ::= \langle \text{标识符} \rangle := \langle \text{表达式} \rangle$

$\langle \text{复合语句} \rangle ::= \text{BEGIN } \langle \text{语句} \rangle \{ ; \langle \text{语句} \rangle \} \text{END}$

$\langle \text{条件} \rangle ::= \langle \text{表达式} \rangle \langle \text{关系运算符} \rangle \langle \text{表达式} \rangle \mid \text{ODD } \langle \text{表达式} \rangle$

$\langle \text{表达式} \rangle ::= [+ \mid -] \langle \text{项} \rangle \{ \langle \text{加法运算符} \rangle \langle \text{项} \rangle \}$

$\langle \text{项} \rangle ::= \langle \text{因子} \rangle \{ \langle \text{乘法运算符} \rangle \langle \text{因子} \rangle \}$

$\langle \text{因子} \rangle ::= \langle \text{标识符} \rangle \mid \langle \text{无符号整数} \rangle \mid ' (' \langle \text{表达式} \rangle ') '$

◇ 例：PL/0 语言的 EBNF 表示

<加法运算符> ::= + | -

<乘法运算符> ::= * | /

<关系运算符> ::= = | # | < | <= | > | >=

<条件语句> ::= IF <条件> THEN <语句>

<过程调用语句> ::= CALL <标识符>

<当型循环语句> ::= WHILE <条件> DO <语句>

<读语句> ::= READ ' (' <标识符> { , <标识符> } ') '

<写语句> ::= WRITE ' (' <表达式> { , <表达式> } ') '

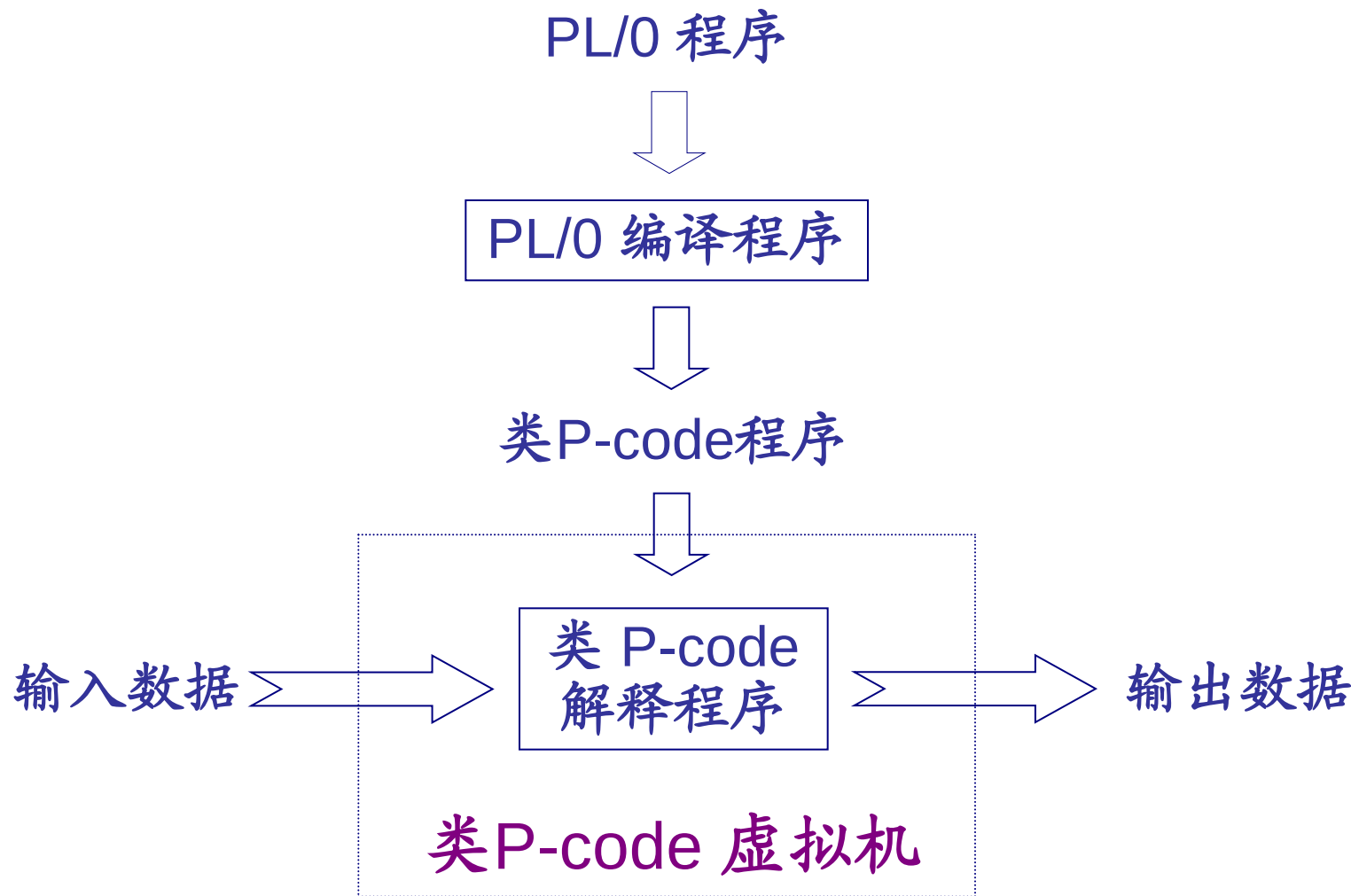
<字母> ::= a | b | ... | X | Y | Z

<数字> ::= 0 | 1 | 2 | ... | 8 | 9

◇ 类型、上下文约束与作用域规则

- 数据类型只有整数类型
- 数据结构只支持简单变量和常数
- 所支持的数字为最长 9 位的十进制数
- 标识符的有效长度为10
- 标识符引用前先要声明
- 过程无参数
- 过程可嵌套，最多嵌套 3 层
- 过程可递归调用
- 内层过程可以引用包围它的外层过程的标识符

类P-code虚拟机



◇ 类 P-code 语言

— 一种栈式机的语言

此类栈式机没有累加器和通用寄存器，有一个栈式存储器，有四个控制寄存器（指令寄存器 I，指令地址寄存器 P，栈顶寄存器 T 和基址寄存器 B），算逻运算都在栈顶进行

◇ 指令格式



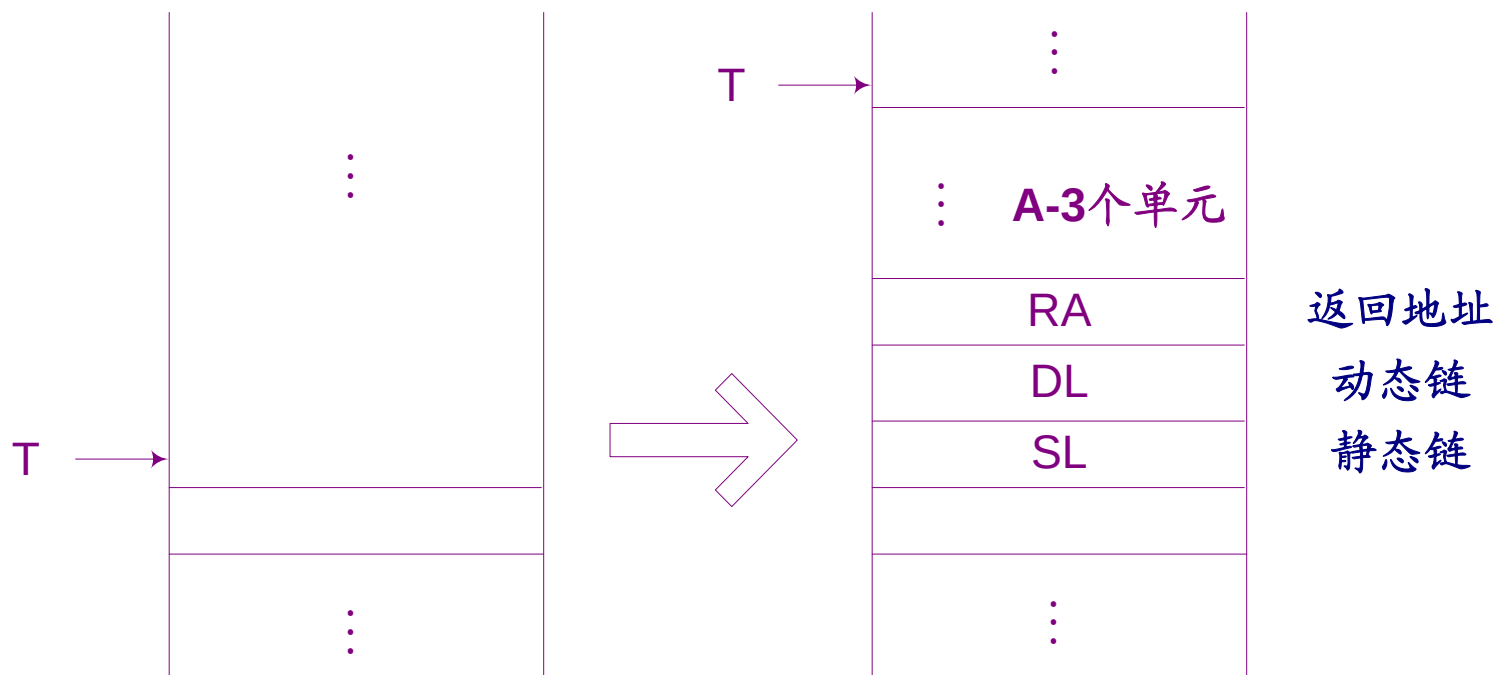
f: 操作码

l: 层次差（标识符引用层减去定义层）

a: 不同的指令含义不同

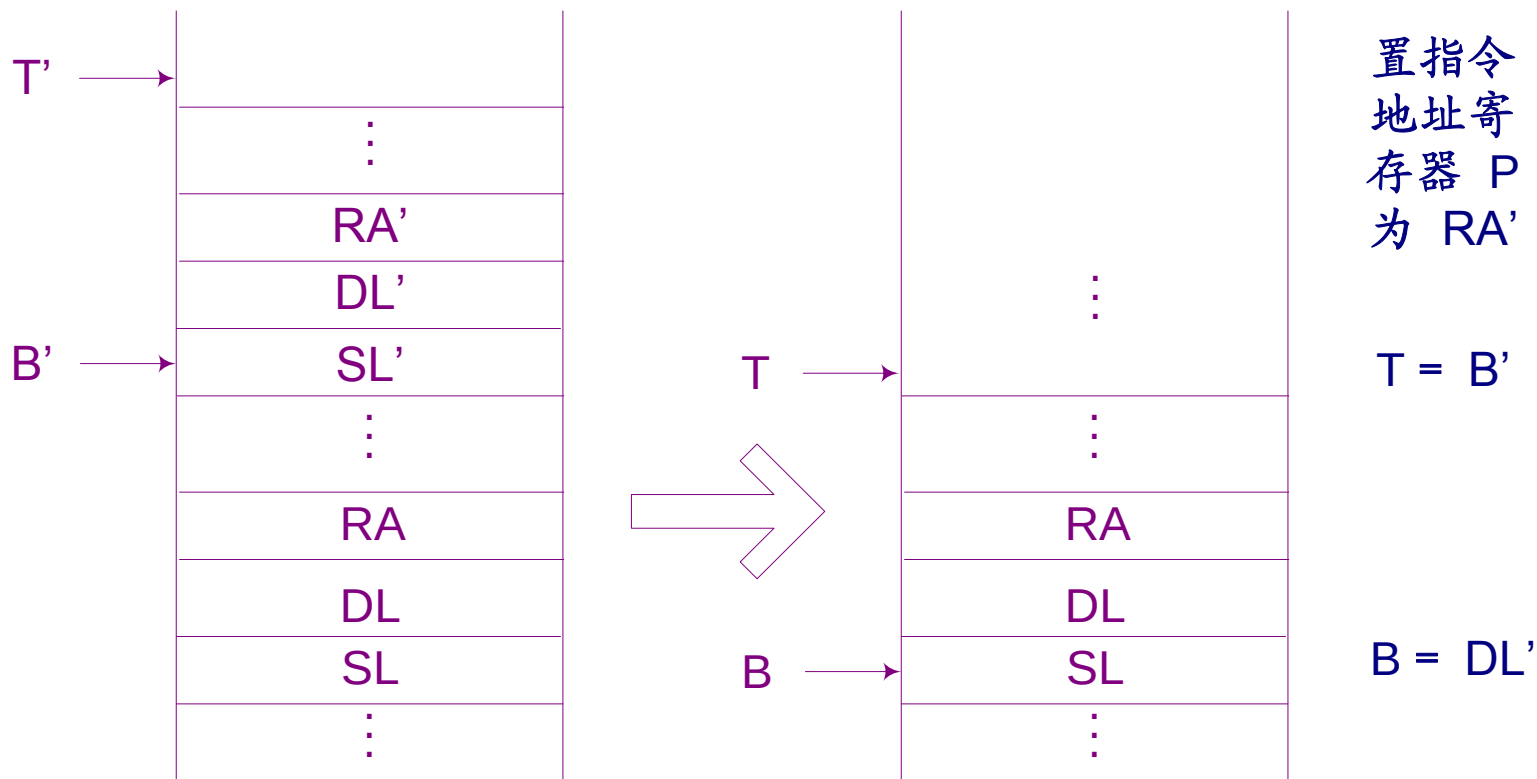
◇ 指令 “INT 0 A”

- 在栈顶开辟 A 个存储单元，服务于被调用的过程
- A 等于该过程的局部变量数加 3
- 3 个特殊的联系单元



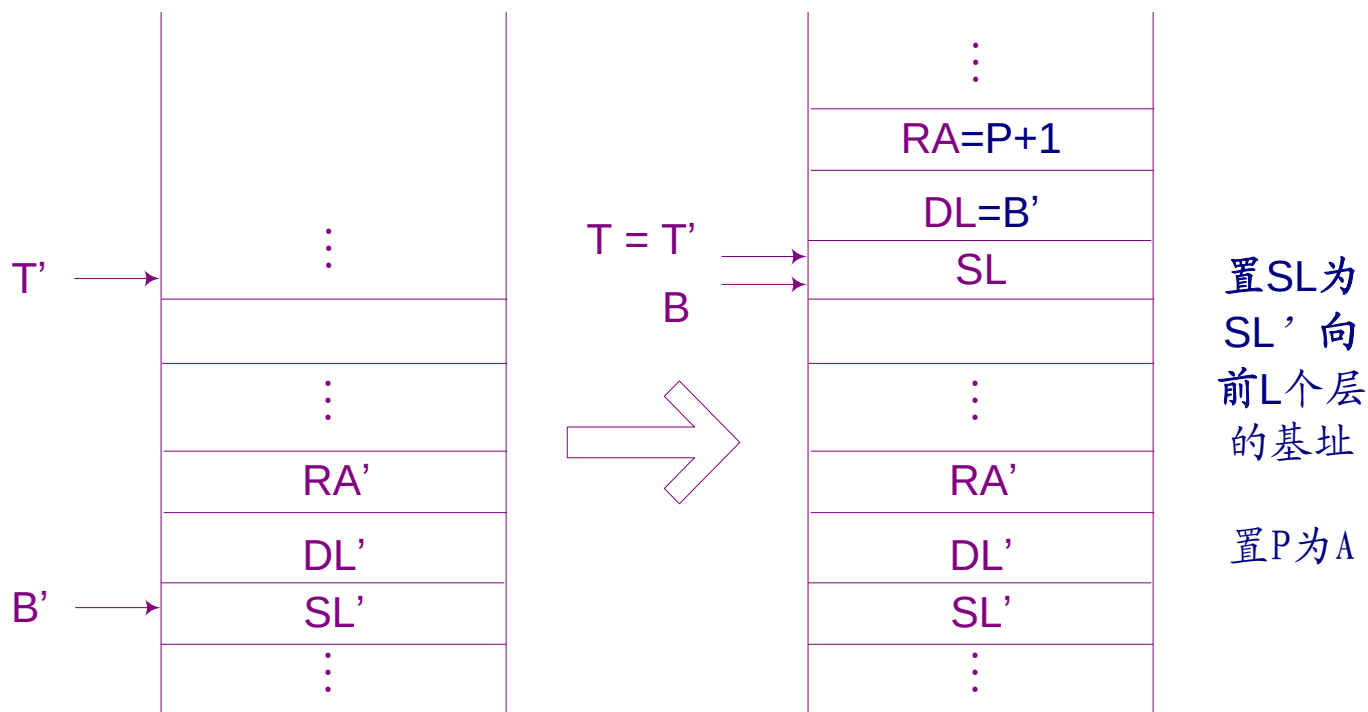
◇ 指令 “OPR 0 0”

- 过程调用结束后, 返回调用点并退栈
- 重置基址寄存器和栈顶寄存器



◇ 指令 “CAL L A”

- 调用地址为 A 的过程（置指令地址寄存器为 A）
- L 为调用过程与被调用过程的层差
- 设置被调用过程的 3 个联系单元



◇ 指令 “LIT 0 A”

- 立即数存入栈顶，即置T所指存储单元的值为A
- T加1

◇ 指令 “LOD L A”

- 将层差为L、偏移量为A的存储单元的值取到栈顶
- T加1

◇ 指令 “STO L A”

- T减1
- 将栈顶的值存入层差为L、偏移量为A的存储单元

◇ **注：**层差为L、偏移量为A的存储单元，即沿当前层静态链SL开始向前第L层的SL作为基址，加上A，即为该单元的地址

◇ 指令 “OPR 0 1”

- 求栈顶元素的相反数，结果值留在栈顶

◇ 指令 “OPR 0 6”

- 栈顶元素的奇偶判断，若为奇数，结果为1；若为偶数，结果为0；结果值留在栈顶

◇ 指令 “OPR 0 2”

- 次栈顶与栈顶的值相加，结果存入次栈顶
- T 减 1

◇ 指令 “OPR 0 3”

- 次栈顶的值减去栈顶的值，结果存入次栈顶
- T 减 1

◇ 指令 “OPR 0 4”

- 次栈顶的值乘以栈顶的值，结果存入次栈顶
- T 减 1

◇ 指令 “OPR 0 5”

- 次栈顶的值除以栈顶的值，结果存入次栈顶
- T 减 1

- ◇ 指令 “OPR 0 8” 比较次栈顶与栈顶是否相等
 - 若相等，结果为0；存结果至次栈顶；T 减1
- ◇ 指令 “OPR 0 9” 比较次栈顶与栈顶是否不相等
 - 若不相等，结果为0；存结果至次栈顶；T 减1
- ◇ 指令 “OPR 0 10” 比较次栈顶是否小于栈顶
 - 若小于，结果为0；存结果至次栈顶；T 减1
- ◇ 指令 “OPR 0 11” 比较次栈顶是否大于等于栈顶
 - 若大于等于，结果为0；存结果至次栈顶；T 减1
- ◇ 指令 “OPR 0 12” 比较次栈顶是否大于栈顶
 - 若大于，结果为0；存结果至次栈顶；T 减1
- ◇ 指令 “OPR 0 13” 比较次栈顶是否小于等于栈顶
 - 若小于等于，结果为0；存结果至次栈顶；T 减1

◇ 指令 “JMP 0 A”

- 无条件转移至地址 A，即置指令地址寄存器为A

◇ 指令 “JPC 0 A”

- 条件转移指令
- 若栈顶为 0，则转移至地址 A，即置指令地址寄存器为A；T 减1

◇ 指令 “OPR 0 14”

- 栈顶的值输出至控制台屏幕
- T 减 1

◇ 指令 “OPR 0 15”

- 控制台屏幕输出一个换行

◇ 指令 “OPR 0 16”

- 从控制台读入一行输入，置入栈顶
- T 加 1

◇ 类P-code 解释程序

— 数据结构

运行栈
指令寄存器

```
int s[stacksize]
struct instruction
{
    enum fct f; /*操作码*/
    int l; /*引用层与声明层的层差*/
    int a; /*因不同的f各异*/
} i
```

指令地址寄存器

```
int p;
```

基址寄存器

```
int b;
```

栈顶寄存器

```
int t;
```

虚拟机代码段

```
struct instruction code[cxmax];
```

◇ 类P-code 解释程序

— 处理流程

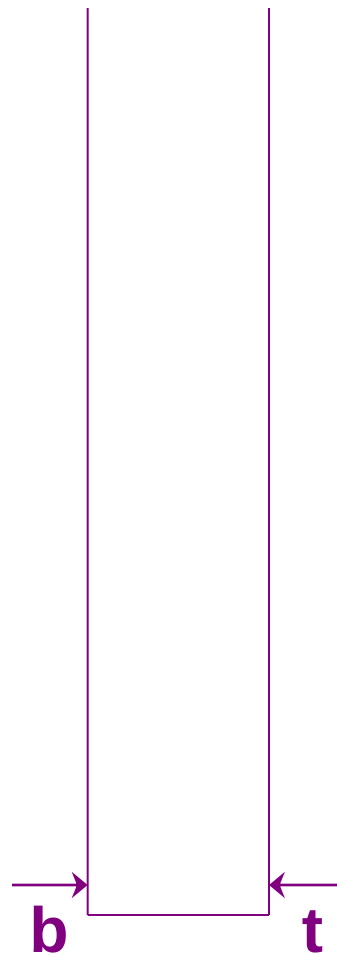
- (1) 初始化 $p=b=t=0$; $s[0]=s[1]=s[2]=0$;
- (2) 取指令到指令寄存器 $i=code[p]$; $p++$;
- (3) 分析并解释执行指令 i
- (4) 若程序未结束 ($p \neq 0$) , 转 (2)
- (5) 返回

类P-code虚拟机

→	(0) jmp 0 8	转向主程序入口
p	(1) jmp 0 2	转向过程p入口
	(2) int 0 3	过程p入口,为过程p开辟空间
	(3) lod 1 3	取变量b的值到栈顶
	(4) lit 0 10	取常数10到栈顶
	(5) opr 0 2	次栈顶与栈顶相加
	(6) sto 1 4	栈顶值送变量c中
	(7) opr 0 0	退栈并返回调用点(16)
	(8) int 0 5	主程序入口开辟5个栈空间
	(9) opr 0 16	从命令行读入值置于栈顶
	(10) sto 0 3	将栈顶值存入变量b中
	(11) lod 0 3	将变量b的值取至栈顶
	(12) lit 0 0	将常数值0进栈
	(13) opr 0 9	次栈顶与栈顶是否不等
	(14) jpc 0 24	相等时转(24) (条件不满足转)
	(15) cal 0 2	调用过程p
	(16) lit 0 2	常数值2进栈
	(17) lod 0 4	将变量c的值取至栈顶
	(18) opr 0 4	次栈顶与栈顶相乘(2*c)
	(19) opr 0 14	栈顶值输出至屏幕
	(20) opr 0 15	换行
	(21) opr 0 16	从命令行读取值到栈顶
	(22) sto 0 3	栈顶值送变量b中
	(23) jmp 0 11	无条件转到循环入口(11)
	(24) opr 0 0	结束退栈

◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

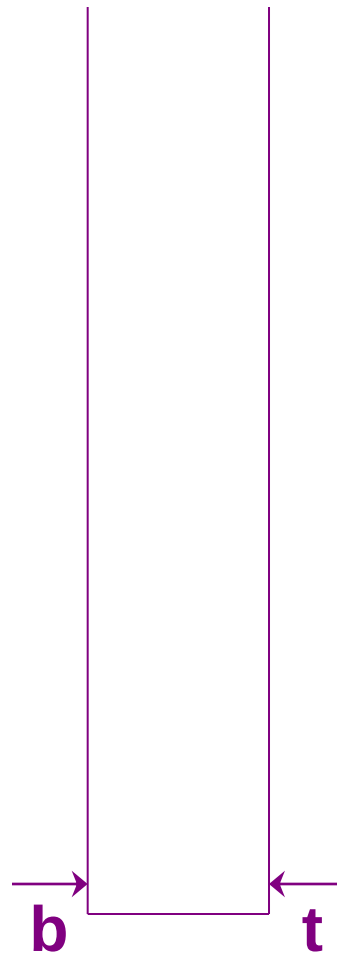


类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
→ (8) int 0 5	主程序入口开辟5个栈空间
p (9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

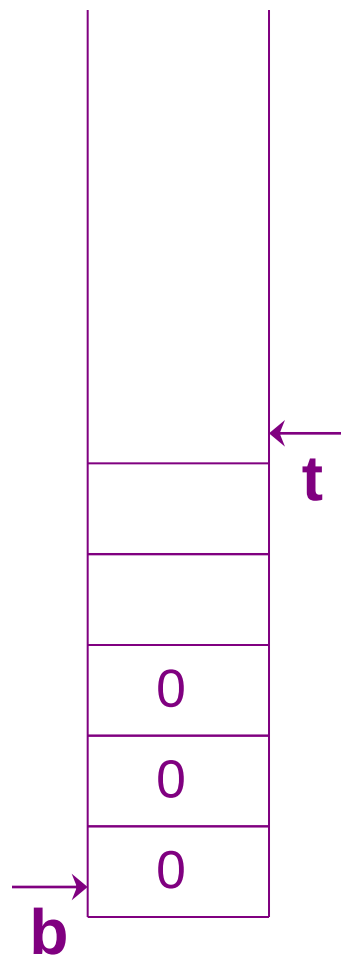
◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```



类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
→ (9) opr 0 16	从命令行读入值置于栈顶
p (10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

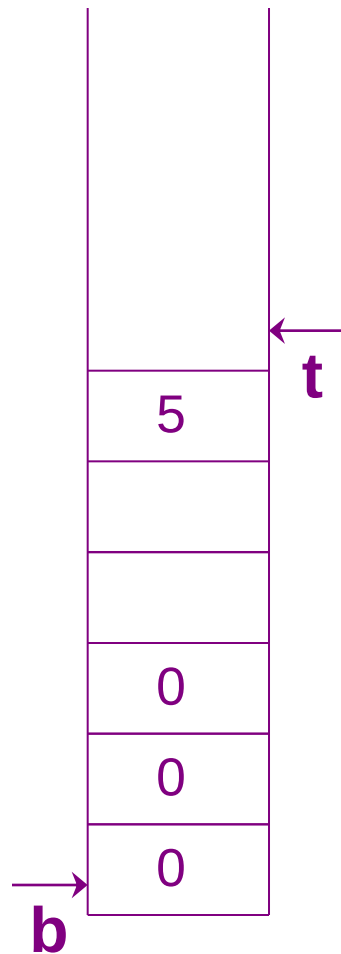


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
    c:=b+a;
end;
begin
    read(b);
    while b#0 do
    begin
        call p;
        write(2*c);
        read(b);
    end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
→ (10) sto 0 3	将栈顶值存入变量b中
p (11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

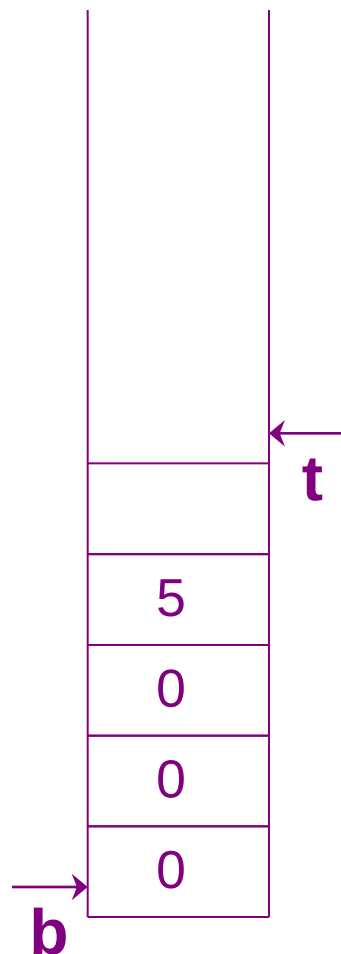


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```


类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
→ (11) lod 0 3	将变量b的值取至栈顶
p (12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

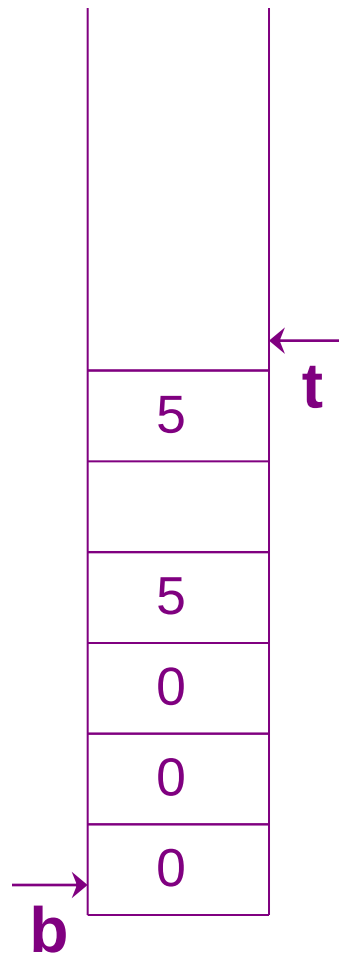


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
→ (12) lit 0 0	将常数值0进栈
p (13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

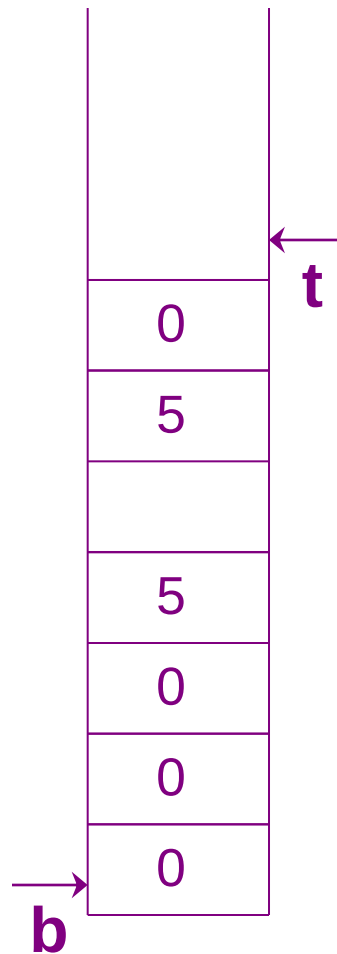


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
→ (13) opr 0 9	次栈顶与栈顶是否不等
p (14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

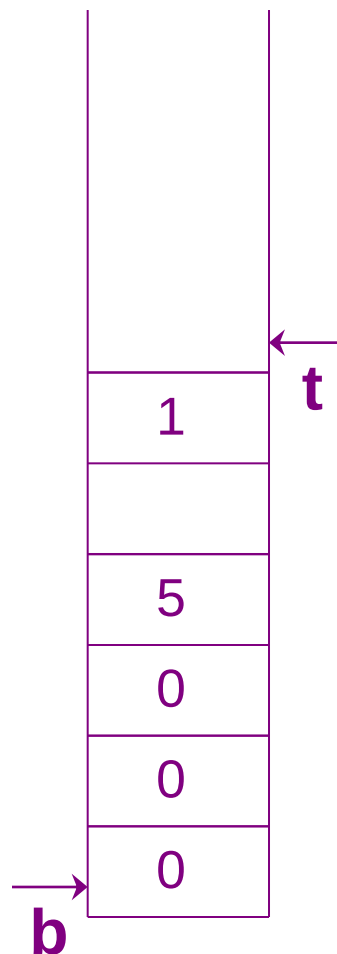


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
→ (14) jpc 0 24	相等时转(24) (条件不满足转)
p (15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

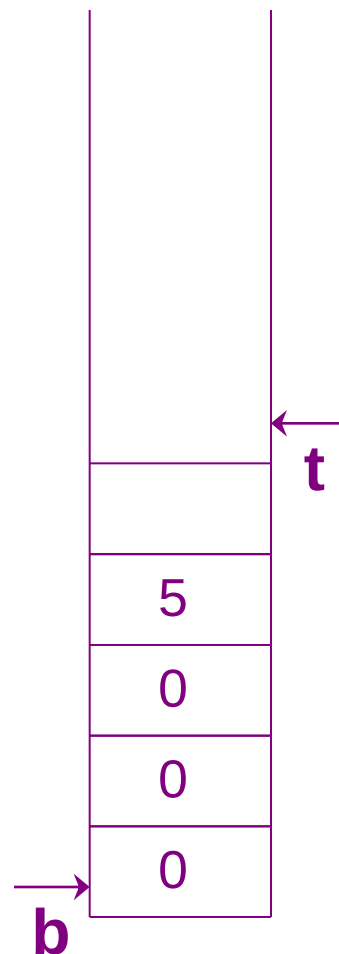


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
→ (15) cal 0 2	调用过程p
p (16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

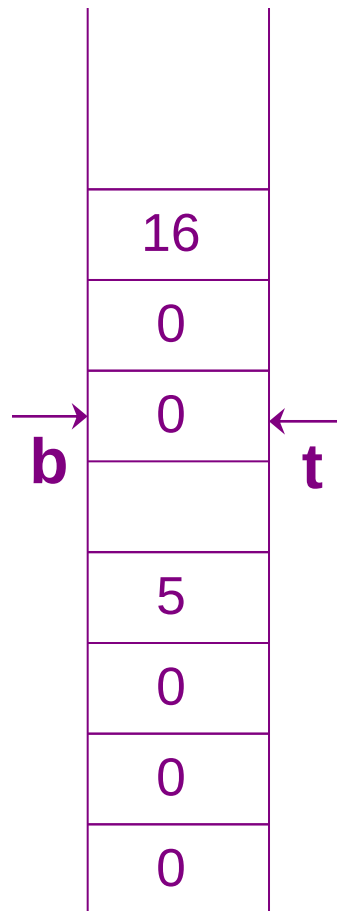


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

	(0) jmp 0 8	转向主程序入口
	(1) jmp 0 2	转向过程p入口
→	(2) int 0 3	过程p入口,为过程p开辟空间
p	(3) lod 1 3	取变量b的值到栈顶
	(4) lit 0 10	取常数10到栈顶
	(5) opr 0 2	次栈顶与栈顶相加
	(6) sto 1 4	栈顶值送变量c中
	(7) opr 0 0	退栈并返回调用点(16)
	(8) int 0 5	主程序入口开辟5个栈空间
	(9) opr 0 16	从命令行读入值置于栈顶
	(10) sto 0 3	将栈顶值存入变量b中
	(11) lod 0 3	将变量b的值取至栈顶
	(12) lit 0 0	将常数值0进栈
	(13) opr 0 9	次栈顶与栈顶是否不等
	(14) jpc 0 24	相等时转(24) (条件不满足转)
	(15) cal 0 2	调用过程p
	(16) lit 0 2	常数值2进栈
	(17) lod 0 4	将变量c的值取至栈顶
	(18) opr 0 4	次栈顶与栈顶相乘(2*c)
	(19) opr 0 14	栈顶值输出至屏幕
	(20) opr 0 15	换行
	(21) opr 0 16	从命令行读取值到栈顶
	(22) sto 0 3	栈顶值送变量b中
	(23) jmp 0 11	无条件转到循环入口(11)
	(24) opr 0 0	结束退栈

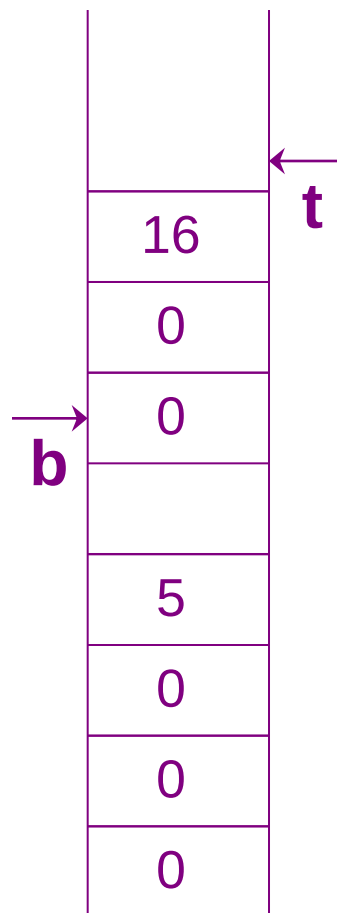


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
    c:=b+a;
end;
begin
    read(b);
    while b#0 do
    begin
        call p;
        write(2*c);
        read(b);
    end
end.
```


✧ 例

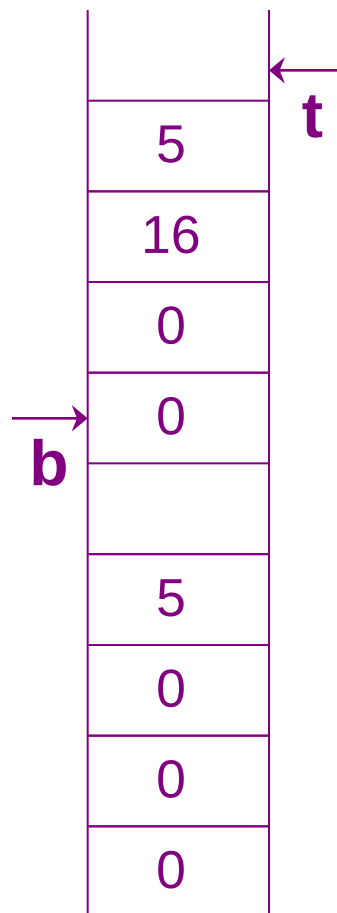
```
const a=10;
var b,c;
procedure p;
begin
    c:=b+a;
end;
begin
    read(b);
    while b#0 do
    begin
        call p;
        write(2*c);
        read(b);
    end
end.
```



(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
→ (3) lod 1 3	取变量b的值到栈顶
p (4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

类P-code虚拟机

	(0) jmp 0 8	转向主程序入口
	(1) jmp 0 2	转向过程p入口
	(2) int 0 3	过程p入口,为过程p开辟空间
	(3) lod 1 3	取变量b的值到栈顶
→	(4) lit 0 10	取常数10到栈顶
p	(5) opr 0 2	次栈顶与栈顶相加
	(6) sto 1 4	栈顶值送变量c中
	(7) opr 0 0	退栈并返回调用点(16)
	(8) int 0 5	主程序入口开辟5个栈空间
	(9) opr 0 16	从命令行读入值置于栈顶
	(10) sto 0 3	将栈顶值存入变量b中
	(11) lod 0 3	将变量b的值取至栈顶
	(12) lit 0 0	将常数值0进栈
	(13) opr 0 9	次栈顶与栈顶是否不等
	(14) jpc 0 24	相等时转(24) (条件不满足转)
	(15) cal 0 2	调用过程p
	(16) lit 0 2	常数值2进栈
	(17) lod 0 4	将变量c的值取至栈顶
	(18) opr 0 4	次栈顶与栈顶相乘(2*c)
	(19) opr 0 14	栈顶值输出至屏幕
	(20) opr 0 15	换行
	(21) opr 0 16	从命令行读取值到栈顶
	(22) sto 0 3	栈顶值送变量b中
	(23) jmp 0 11	无条件转到循环入口(11)
	(24) opr 0 0	结束退栈

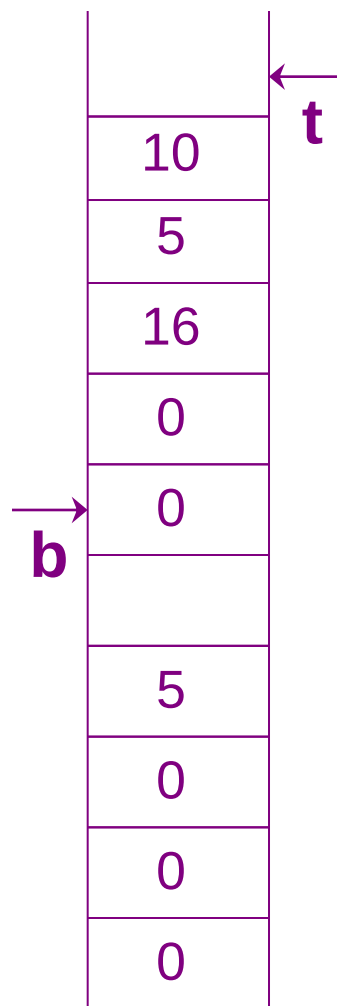


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
    c:=b+a;
end;
begin
    read(b);
    while b#0 do
    begin
        call p;
        write(2*c);
        read(b);
    end
end.
```


类P-code虚拟机

→	(0) jmp 0 8	转向主程序入口
	(1) jmp 0 2	转向过程p入口
	(2) int 0 3	过程p入口,为过程p开辟空间
	(3) lod 1 3	取变量b的值到栈顶
	(4) lit 0 10	取常数10到栈顶
→	(5) opr 0 2	次栈顶与栈顶相加
p	(6) sto 1 4	栈顶值送变量c中
	(7) opr 0 0	退栈并返回调用点(16)
	(8) int 0 5	主程序入口开辟5个栈空间
	(9) opr 0 16	从命令行读入值置于栈顶
	(10) sto 0 3	将栈顶值存入变量b中
	(11) lod 0 3	将变量b的值取至栈顶
	(12) lit 0 0	将常数值0进栈
	(13) opr 0 9	次栈顶与栈顶是否不等
	(14) jpc 0 24	相等时转(24) (条件不满足转)
	(15) cal 0 2	调用过程p
	(16) lit 0 2	常数值2进栈
	(17) lod 0 4	将变量c的值取至栈顶
	(18) opr 0 4	次栈顶与栈顶相乘(2*c)
	(19) opr 0 14	栈顶值输出至屏幕
	(20) opr 0 15	换行
	(21) opr 0 16	从命令行读取值到栈顶
	(22) sto 0 3	栈顶值送变量b中
	(23) jmp 0 11	无条件转到循环入口(11)
	(24) opr 0 0	结束退栈

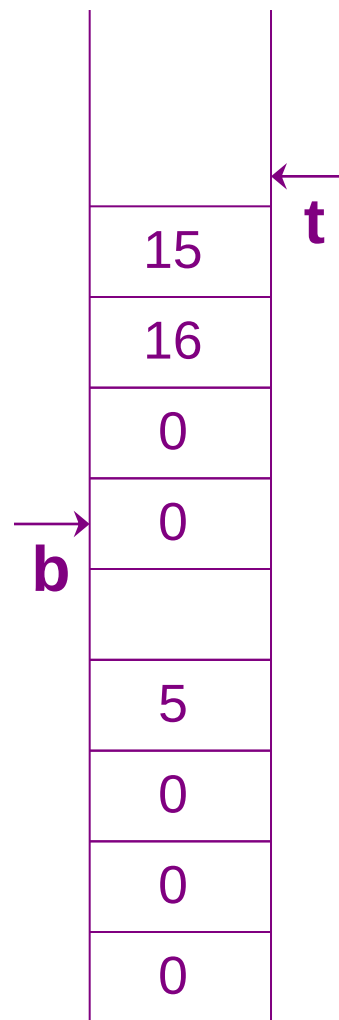


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
    c:=b+a;
end;
begin
    read(b);
    while b#0 do
    begin
        call p;
        write(2*c);
        read(b);
    end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
→ (6) sto 1 4	栈顶值送变量c中
p (7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

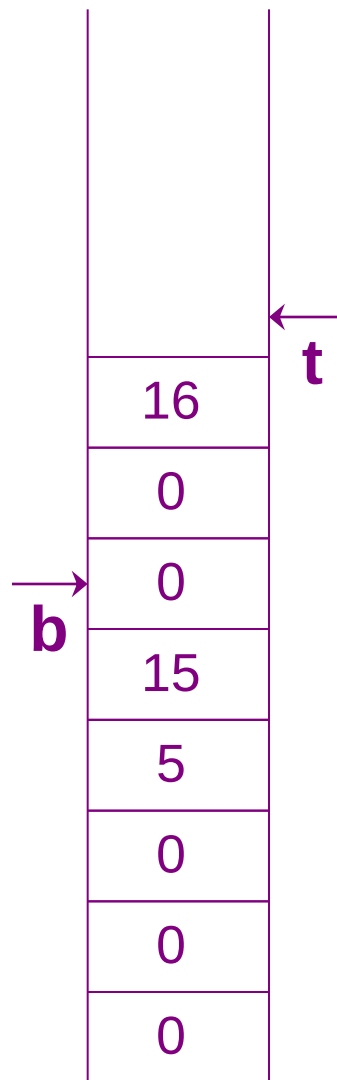


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

	(0) jmp 0 8	转向主程序入口
	(1) jmp 0 2	转向过程p入口
	(2) int 0 3	过程p入口,为过程p开辟空间
	(3) lod 1 3	取变量b的值到栈顶
	(4) lit 0 10	取常数10到栈顶
	(5) opr 0 2	次栈顶与栈顶相加
	(6) sto 1 4	栈顶值送变量c中
→	(7) opr 0 0	退栈并返回调用点(16)
p	(8) int 0 5	主程序入口开辟5个栈空间
	(9) opr 0 16	从命令行读入值置于栈顶
	(10) sto 0 3	将栈顶值存入变量b中
	(11) lod 0 3	将变量b的值取至栈顶
	(12) lit 0 0	将常数值0进栈
	(13) opr 0 9	次栈顶与栈顶是否不等
	(14) jpc 0 24	相等时转(24) (条件不满足转)
	(15) cal 0 2	调用过程p
	(16) lit 0 2	常数值2进栈
	(17) lod 0 4	将变量c的值取至栈顶
	(18) opr 0 4	次栈顶与栈顶相乘(2*c)
	(19) opr 0 14	栈顶值输出至屏幕
	(20) opr 0 15	换行
	(21) opr 0 16	从命令行读取值到栈顶
	(22) sto 0 3	栈顶值送变量b中
	(23) jmp 0 11	无条件转到循环入口(11)
	(24) opr 0 0	结束退栈

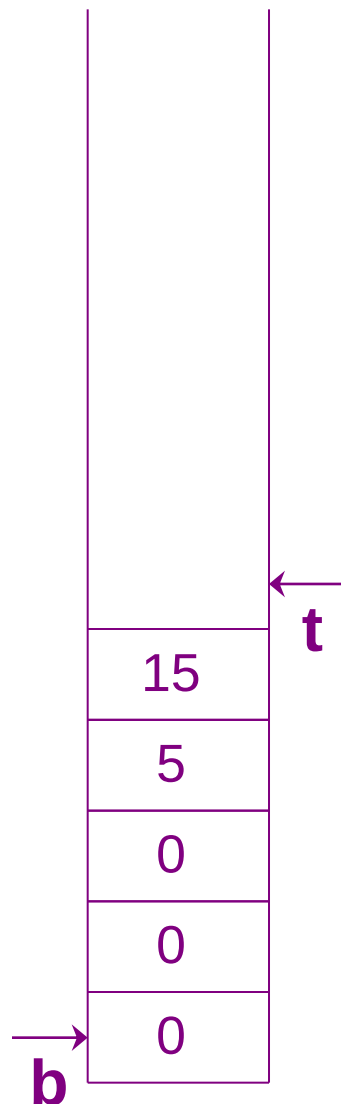


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
→ (16) lit 0 2	常数值2进栈
p (17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

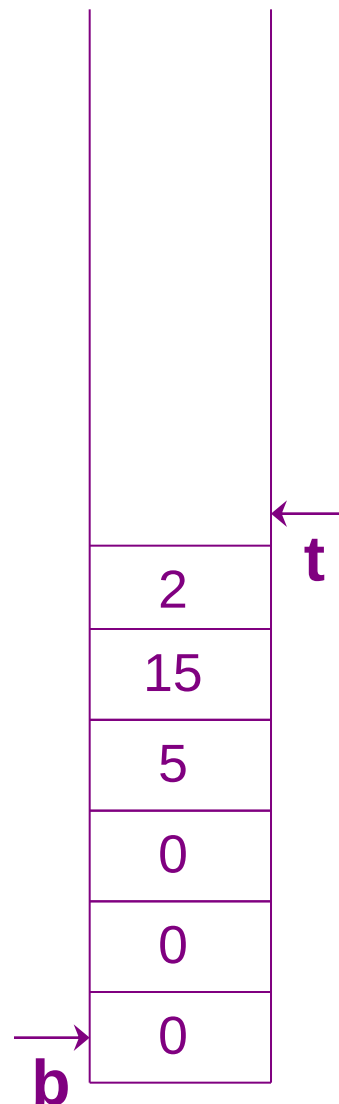


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
→ (17) lod 0 4	将变量c的值取至栈顶
p (18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

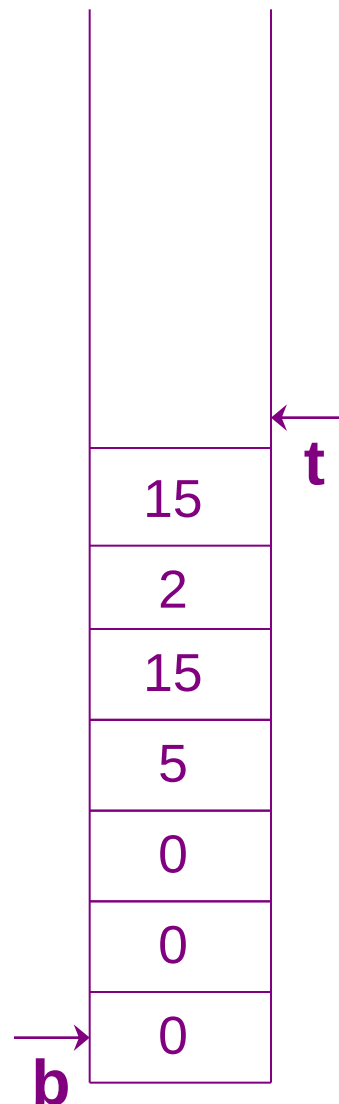


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

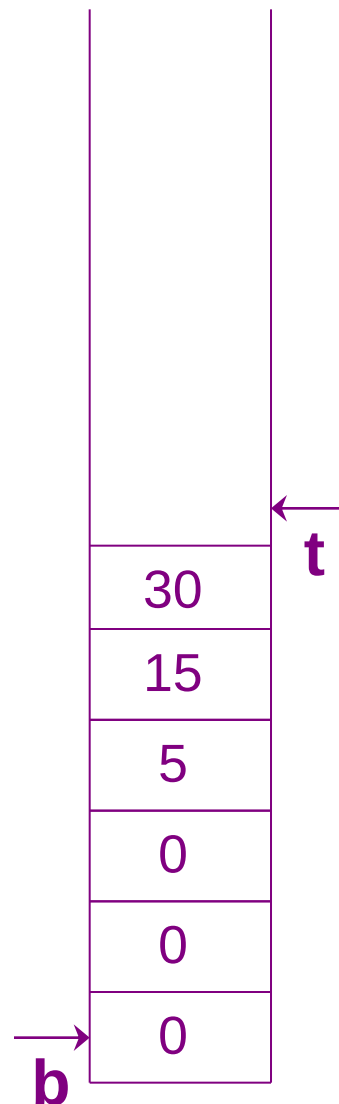


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```


类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

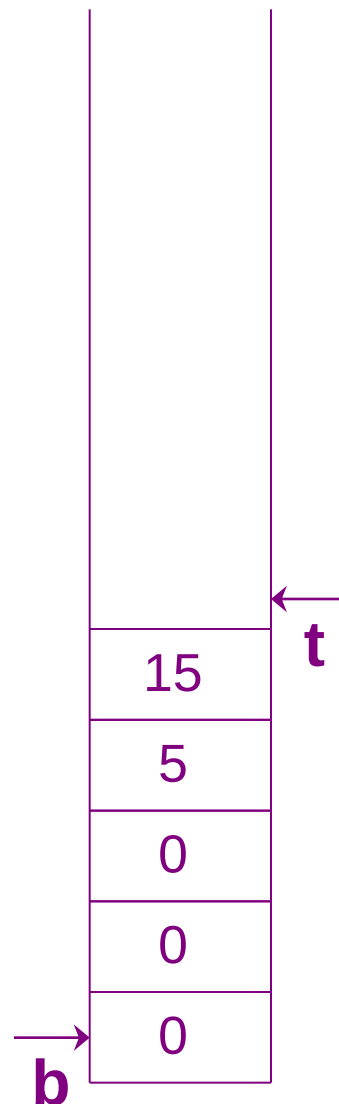


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
→ (20) opr 0 15	换行
p (21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

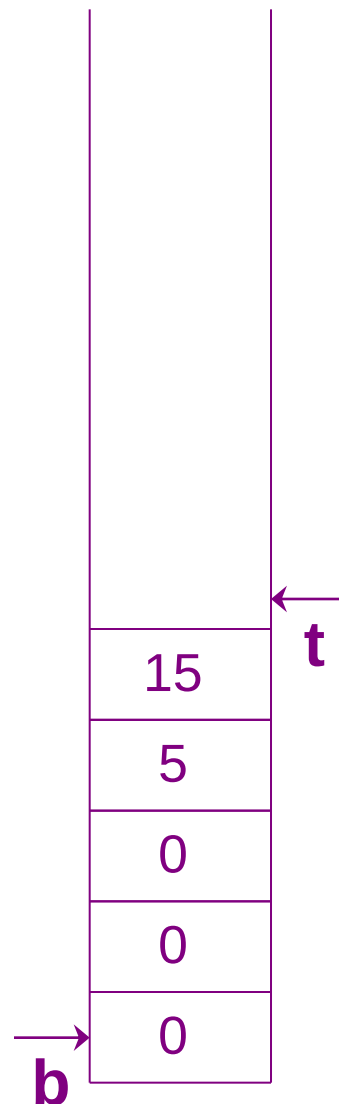


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```


类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

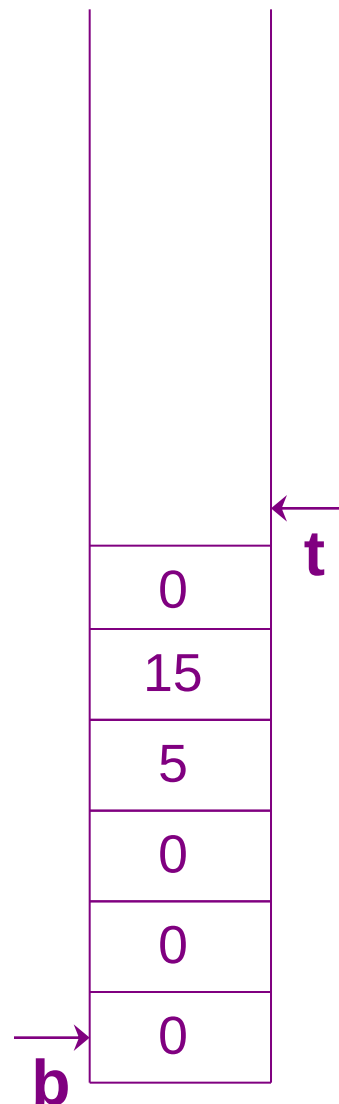


◇ 例

```
const a=10;
var b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

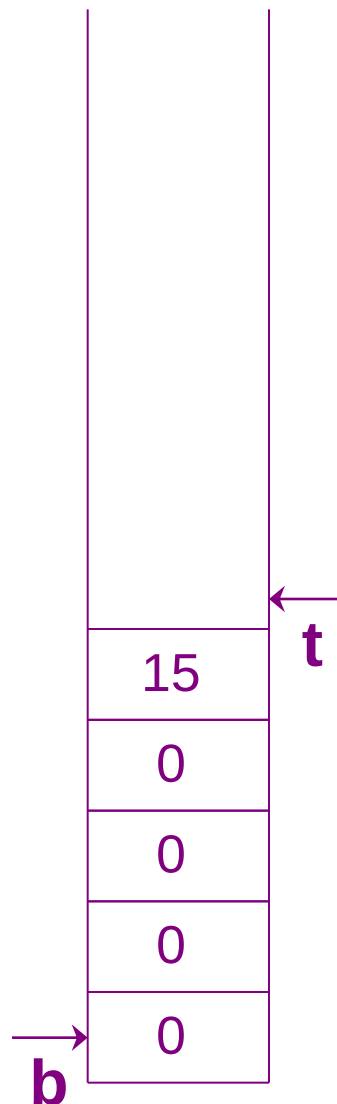


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

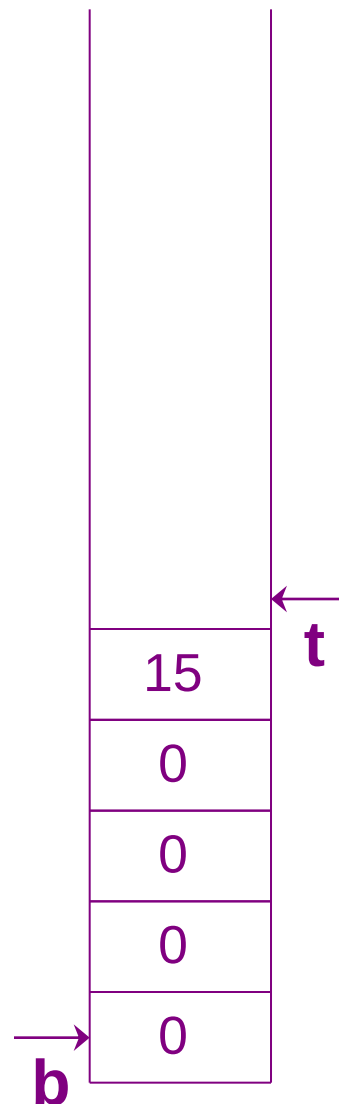


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
→ (11) lod 0 3	将变量b的值取至栈顶
p (12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

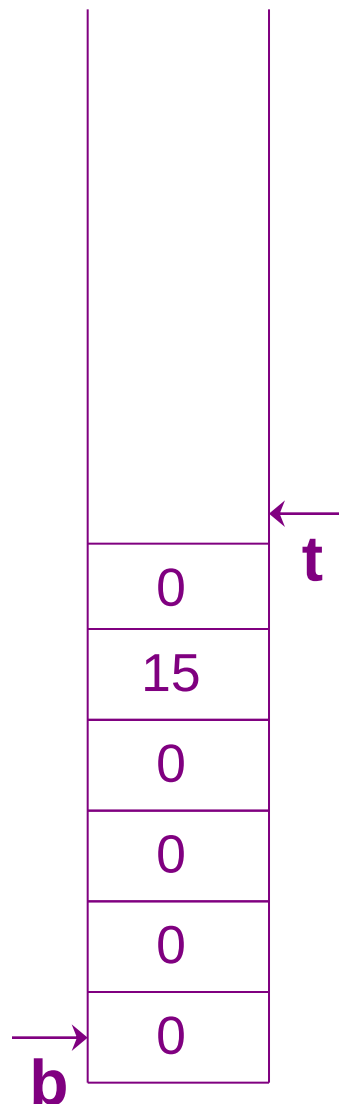


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
→ (12) lit 0 0	将常数值0进栈
p (13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

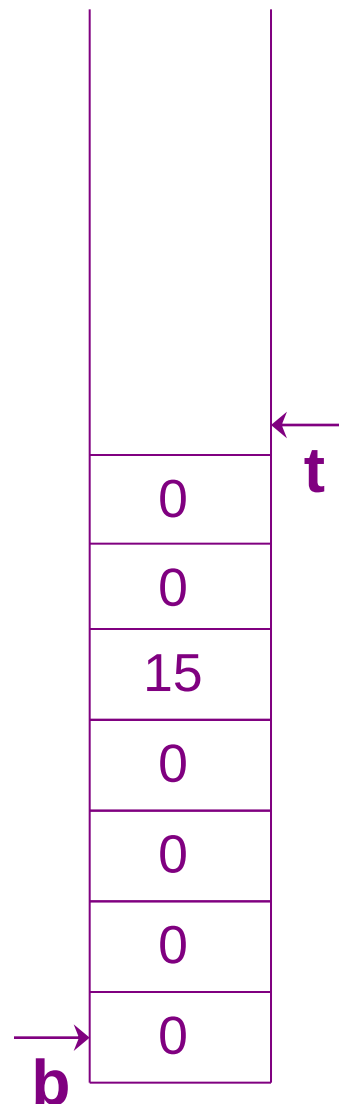


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
→ (13) opr 0 9	次栈顶与栈顶是否不等
p (14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

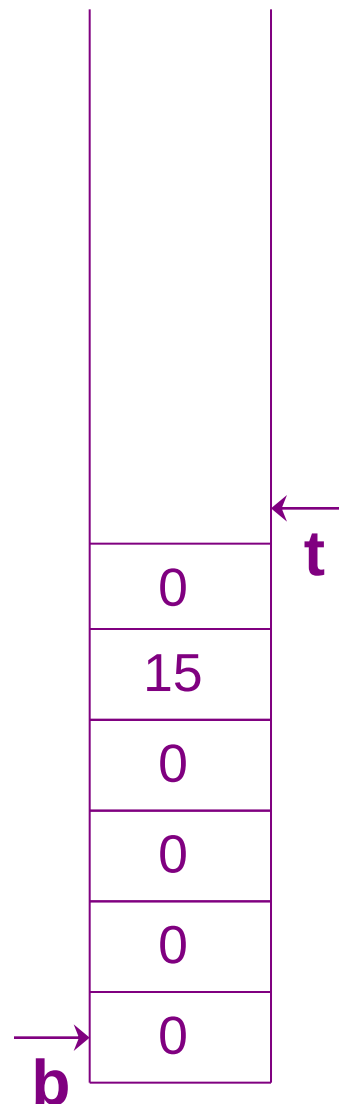


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```


类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
→ (14) jpc 0 24	相等时转(24) (条件不满足转)
p (15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

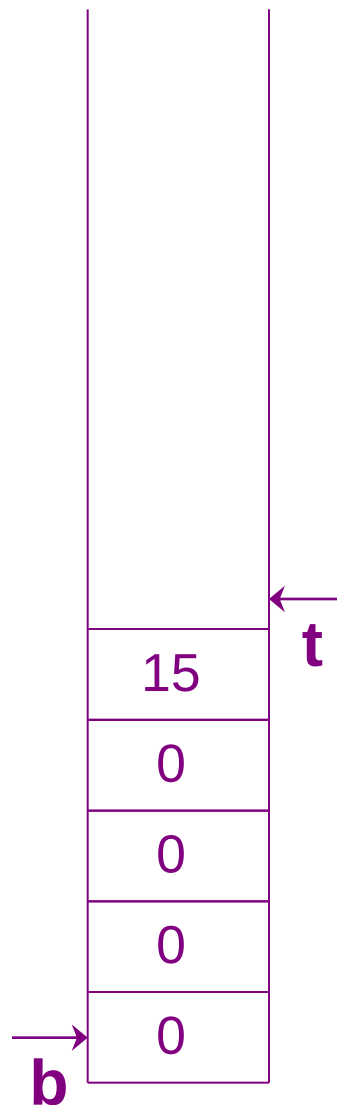


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

类P-code虚拟机

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

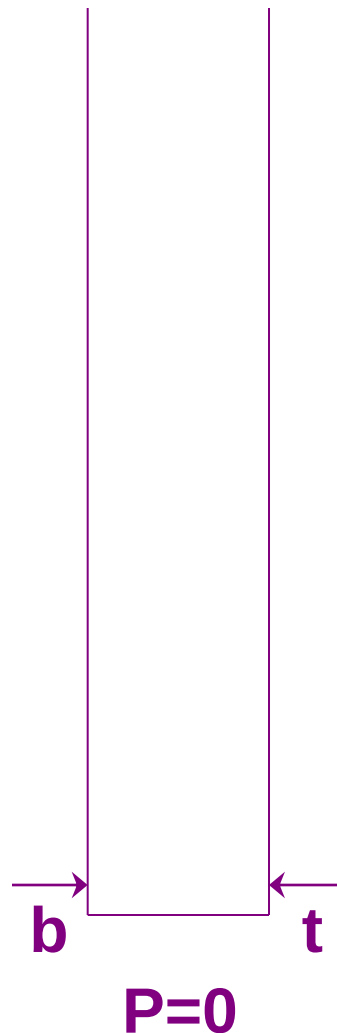


◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```


类P-code虚拟机

→ (0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈



◇ 例

```
const a=10;
var  b,c;
procedure p;
begin
  c:=b+a;
end;
begin
  read(b);
  while b#0 do
  begin
    call p;
    write(2*c);
    read(b);
  end
end.
```

◇ 词法分析子程序

- int getsym()

◇ 功能

- 从当前输入符号起扫描下一个词法单位，即单词

- 通过下列全局变量传递单词的类别

enum symbol sym;

- 通过下列全局变量传递标识符单词的值

char id [al+1]; /*al为标识符的最大长度*/

- 通过下列全局变量传递常量单词的值

int num;

◇ 单词类别

— 保留字

BEGIN、END、IF、THEN、.....

— 运算符

+、-、*、/、:=、=、#、<=、>=、.....

— 标识符

自定义的变量名、常数名、过程名

— 常数

如: 10, 25, 100等整数

— 界符

如: ‘,’、‘.’、‘;’、‘(’、
‘)’、.....

☆ 词法分析子程序 getsym() 的处理流程

- 从源程序扫描下一个字符 (调用 getch 子程序)
- 忽略空格、换行和 TAB (每忽略一个, 扫描下一个)
enum symbol sym;
- 识别单词 (每扫描过一个字符, 调用一次 getch 子程序)
 - 识别保留字
 - 识别标识符
 - 拼数
 - 识别单字符单词
 - 拼双字符单词
- 根据单词类别设置 sym, id 和 num

◇ 单词的识别

- 可以借助于状态转换图进行设计
- 状态转换图类似于有限状态自动机
- 识别标识符单词、数字单词、单字符单词和双字符单词的状态转换图见下页
- 保留字单词的识别可以在识别标识符单词的基础上，再去检索保留字表

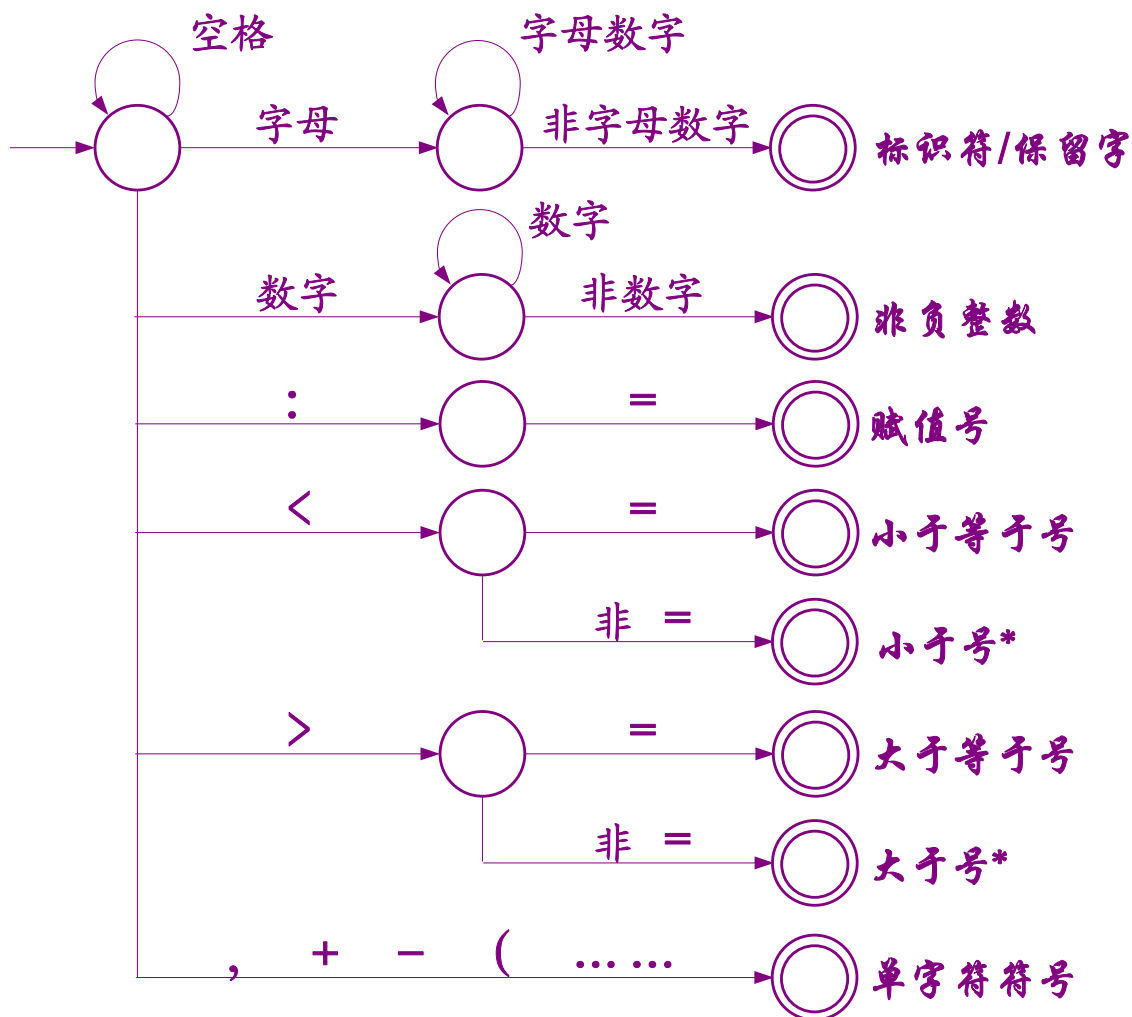
PL/0 编译程序的词法分析



清华大学

《编译原理》

◇ 实现单词识别的状态转换图



◇ 功能

- 分析 PL/0 源程序的单词流是否符合 PL/0 语法
- 报告语法错误
- 产生源程序的语法分析结果，以语法分析树或与之等价的形式体现出来

◇ 分析方法

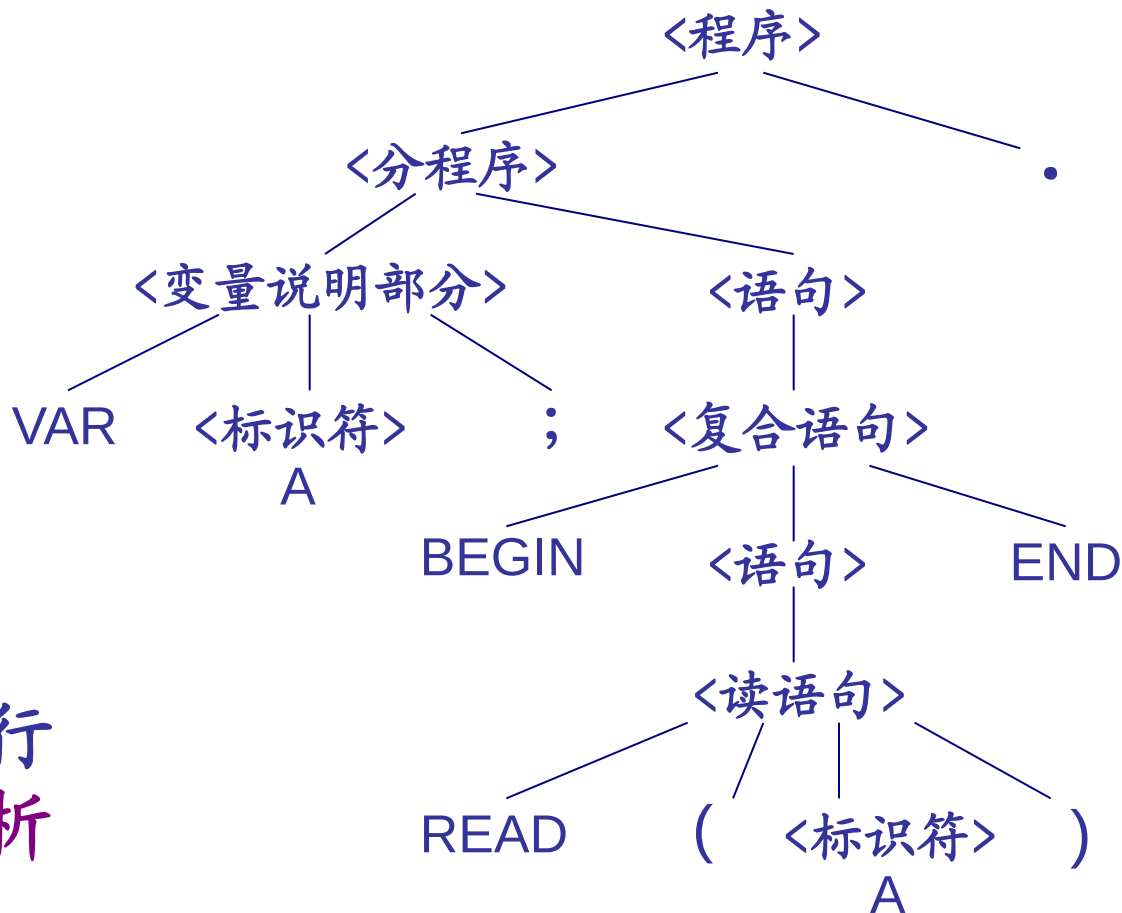
- 借助于PL/0的语法描述图或EBNF表示进行自顶向下分析

合法的PL/0程序都可以对应一棵自顶向下构造的语法分析树

语法分析树的根节点为<程序>，叶节点为构成源程序的单词，每个内部节点代表构成源程序的各种不同的语法单位

◇ 自顶向下分析举例

VAR A;
BEGIN
 READ(A)
END.



自顶向下进行
递归下降分析

◇ 实现方法

— 递归子程序法

可以自然实现递归下降分析过程

每个语法单位都对应一个分析子程序，其设计基于该语法单位的语法描述图或EBNF表示

递归下降分析过程从调用语法单位<程序>对应的子程序开始，运行时的调用关系反映了语法分析树的结构

◇ 递归子程序的设计

- 沿语法分析图箭头所指方向进行如下工作：
遇到一个语法单元，调用相应的子程序
遇到一个词法单位，则判断当前读入的单词是否与该词法单位相匹配，若匹配，再读取下一个单词继续分析；若不匹配，则进行出错处理
- 利用EBNF的方法与此相似

◇ 递归子程序的设计实例

– $\langle \text{表达式} \rangle ::= [+|-] \langle \text{项} \rangle \{ (+|-) \langle \text{项} \rangle \}$

```
int expression(...)
{
    if (sym==plus || sym==minus)      /* 此时表达式被看作正的或负的项 */
        getsym();
        term (...);    /* 处理<项> */
    }
    else /* 此时表达式被看作项的加减 */
    {
        term (...);    /* 处理<项> */
    }
    while (sym==plus || sym==minus)
    {
        getsym;
        term (...);    /* 处理<项> */
    }
    return 0;
}
```

◇ 递归子程序的设计实例

– $\langle \text{项} \rangle ::= \langle \text{因子} \rangle \{ (* | /) \langle \text{因子} \rangle \}$

```
int term(...)
{
    factor (...);          /*处理<因子>*/
    while (sym==times || sym==slash)
    {
        getsym();
        factor (...);      /* 处理<因子> */
    }
    return 0;
}
```

◇ 递归子程序的设计实例

– $\langle \text{因子} \rangle ::= \langle \text{标识符} \rangle \mid \langle \text{无符号整数} \rangle \mid ' (' \langle \text{表达式} \rangle$

$\mid ')'$

```
int factor (...)
```

```
{
```

```
    if (sym==ident)          /* <因子>为常量或变量 */
```

```
        getsym();
```

```
    else if (sym==number)    /* <因子>为立即数 */
```

```
        getsym();
```

```
    else if (sym==lparen);   /* <因子>为立即数 */
```

```
    {
```

```
        expression(...);
```

```
        if (sym==rparen)
```

```
            getsym();
```

```
        else
```

```
            error(22); /*提示22号出错信息: 缺少右括号*/
```

```
    }
```

```
    return 0;
```

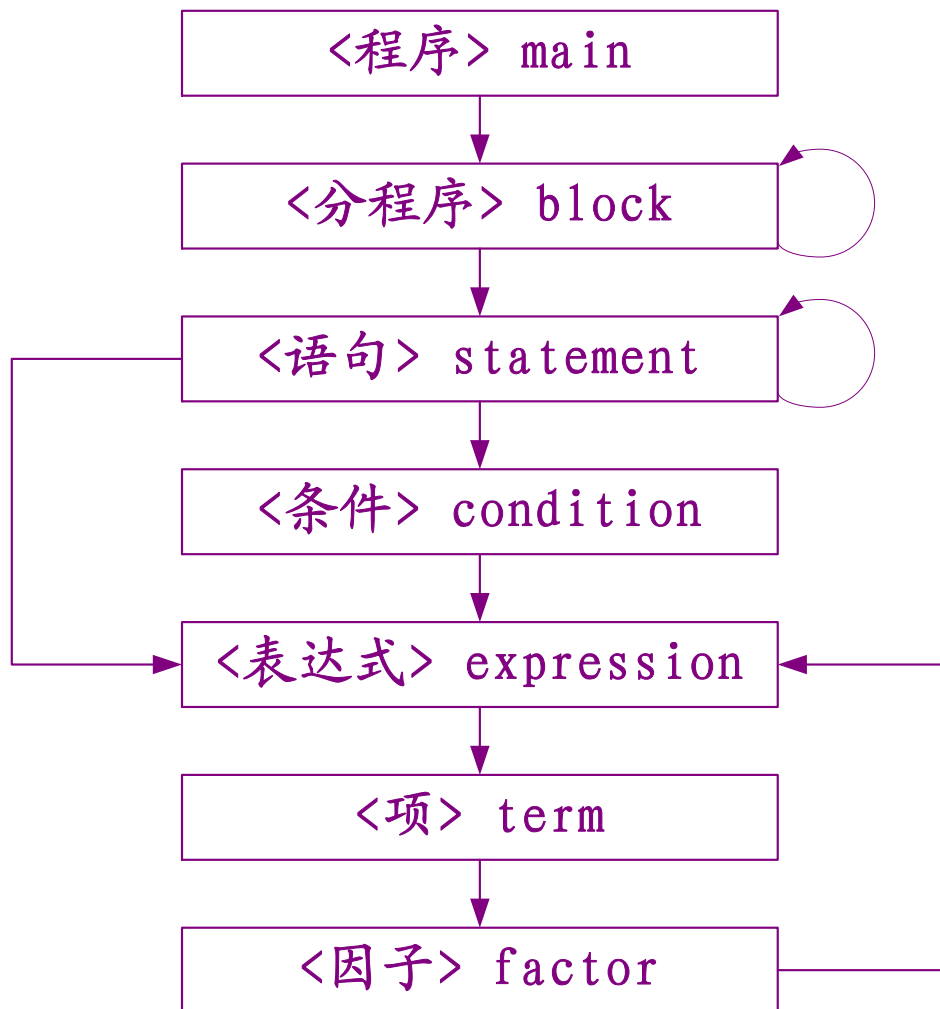
```
}
```

☆ PL/0 语法分析程序入口

– $\langle \text{程序} \rangle ::= \langle \text{分程序} \rangle .$

```
int main()
{
    ... /*初始化*/
    ... /*读写文件*/
    getsym();
    block(...)    /* 处理<分程序>*/
    ...
    if (sym != period)
        error(9); /*提示 9 号出错信息: 缺少程序结束符'.'*/
    ...
    return 0;
}
```

◇ 主要语法单位相应子程序之间的调用关系



◇ 功能

- 借助符号表进行上下文相关的静态语义分析
- 确保符号表可以体现作用域规则
- 确保标识符属性与上下文环境一致
- 确保标识符先声明后引用
- 确保标识符的长度、数字的位数、过程嵌套说明的层数符合PL/0语言的约定
- 提示语义错误信息

◇ 符号表结构

```
Enum object {  
    constant,  
    variable,  
    procedur  
};
```

```
Struct tablestruct {  
    char name[a1];      /*a1-名字最大长度*/  
    enum object kind;  
    int val;             /*constant标识符的数值*/  
    int level;           /*标识符所在的层, constant标识符不用*/  
    int adr;             /*标识符相对地址, constant标识符不用*/  
    int size;            /*需分配的数据区大小, 仅procedur标识符用到*/  
};
```

```
Struct tablestruct table[txmax];  /*txmax-符号表容量*/
```

PL/0 编译程序的语义分析



清华大学

《编译原理》

◇ 符号表结构

◇ 例：右边程序的说明
部分分析后的符
号表如下所示

```
CONST A=35, B=49;  
VAR C, D, E;  
PROCEDURE P;  
    VAR G, X, Y, Z;
```

NAME: A	KIND: CONSTANT	VAL: 35		
NAME: B	KIND: CONSTANT	VAL: 49		
NAME: C	KIND: VARIABLE	LEVEL: LEV	ADDR: DX	
NAME: D	KIND: VARIABLE	LEVEL: LEV	ADDR: DX+1	
NAME: E	KIND: VARIABLE	LEVEL: LEV	ADDR: DX+2	
NAME: P	KIND: PROCEDUR	LEVEL: LEV	ADDR:	SIZE: 7
NAME: G	KIND: VARIABLE	LEVEL: LEV+1	ADDR: DX	
.....				

PL/0 编译程序的语义分析



清华大学

《编译原理》

✧ 符号表与作用域规则

✧ 例：右边程序在处理到/*here*/时的符号表如下所示

NAME	KIND	VAL / LEVEL	ADDR	SIZE
a	CONSTANT	25		
x	VARIABLE	LEV	DX	
y	VARIABLE	LEV	DX+1	
p	PROCEDUR	LEV		4
r	PROCEDUR	LEV		5
x	VARIABLE	LEV+1	DX	
s	VARIABLE	LEV+1	DX+1	
t	PROCEDUR	LEV+1		4

```
const a=25;
var x,y;
procedure p;
  var z;
  begin
    .....
  end;
procedure r;
  var x, s;
  procedure t;
    var v;
    begin
      .....
    end;
  begin /*here*/
    .....
  end.
```

◇ 符号表管理

— 登录 (在符号表中插入一项)

```
void enter(enum object k, int* ptx, int lev, int* pdx)
```

```
/* k: 名字种类 const,var or procedure
```

```
   ptx: 名字表尾指针的指针
```

```
   lev: 名字所在的层次
```

```
   pdx: 当前应分配变量的相对地址, 分配后增加1
```

```
*/
```

— 查询

```
int position(char * idt, int tx)
```

```
/* idt: 被查标识符名字串
```

```
   tx: 符号表栈当前栈顶的位置
```

```
返回所查标识符在符号表栈中的位置, 没查到则返回0*/
```

◇ 语义处理举例 —— 变量声明语句的处理

<变量说明部分> ::= var <标识符> {, <标识符>};

```
if (sym == varsym){           /* 收到变量声明符号，开始处理变量声明 */
    getsymdo;                 /*调用 getsym ( )的宏*/
    do {
        vardeclarationdo(&tx, lev, &dx);      /*见下页*/
        while (sym == comma) {
            getsymdo;
            vardeclarationdo(&tx, lev, &dx);
        }
        if (sym == semicolon){
            getsymdo;
        }else error(5);
    } while (sym == ident);
}
```

◇ 语义处理举例 —— 变量声明语句的处理 (接上页)

```
int vardeclaration(int* ptx,int lev,int* pdx)
```

```
    /* ptx: 符号表尾位置; lev: 当前层; pdx: 在当前层的偏移量; */
```

```
{
```

```
    if (sym == ident){
```

```
        enter(variable, ptx, lev, pdx); //填写名字表
```

```
        getsymdo;
```

```
    } else
```

```
    {
```

```
        error(4);    /* var 后应是标识符 */
```

```
    }
```

```
    return 0;
```

```
}
```

◇ 语义处理举例 —— 变量引用的处理

- 当遇到标识符的引用时就调用POSITION函数查符号表，看是否有过正确定义，若已有，则从表中取相应的有关信息，供代码的生成使用；若无定义则报错
- 若在符号表有过正确定义，检查引用与说明的属性是否一致，若不一致则报错

◇ 语义处理举例 — 变量引用的处理 (以赋值语句中的变量引用为例)

```
if (sym == ident){ /* 准备按照赋值语句处理 */
    i = position(id, *ptx);
    if (i == 0){
        error(11); /* 变量未找到 */
    }else
    {
        if (table[i].kind != variable) {
            error(12); /* 赋值语句格式错误 */
            i = 0;
        }else
        {
            .....
            gendo(sto, ...); /*生成目标代码*/
            .....
        }
    }
}
```

✧ 错误处理的原则

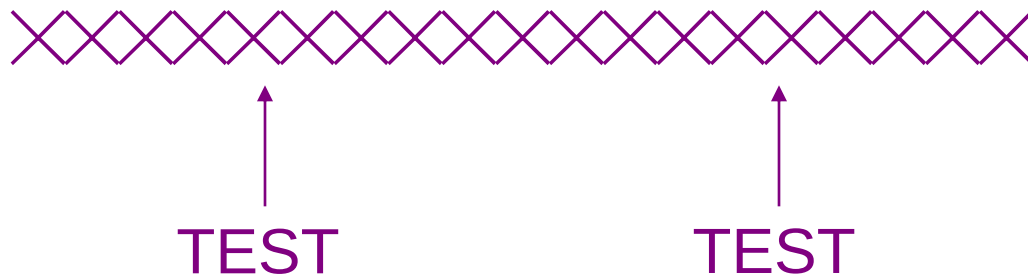
- 尽可能准确指出错误位置和错误属性
- 尽可能进行校正

✧ PL/0 编译程序的错误处理

- 短语层恢复 (phrase-level error recovery)

◇ 短语层恢复

- 在进入某个语法单位时，调用TEST函数，检查当前符号是否属于该语法单位的开始符号集合。若不属于，则滤去开始符号和后跟符号集合外的所有符号
- 在语法单位分析结束时，调用TEST函数，检查当前符号是否属于调用该语法单位时应有的后跟符号集合。若不属于，则滤去后跟符号和开始符号集合外的所有符号



PL/0 编译程序的错误处理



清华大学

《编译原理》

◇ 开始符号集合与后跟符号集合

语法单位	开始符号集合	后跟符号集合
分程序	const var procedure ident l f call begin while read write	. ;
语句	ident call begin if while read write	. ; end
条件	odd + - (ident number	Then do
表达式	+ - (ident number	. ;) rop end then do
项	ident number (	. ;) rop + - end then do
因子	ident number (	. ;) rop + - * / end then do

✧ test函数

```
int test(bool* s1, bool* s2, int n)
/* s1-需要的集合, s2-补救的集合 */
{
    if (!inset(sym, s1))
    {
        error(n);
        while ((!inset(sym,s1)) && (!inset(sym,s2)))
        {
            getsymdo;
        }
    }
    return 0;
}
```

◇ 例：因子的处理过程

```
int factor(bool* fsys, ...) /* fsys-后跟符号集合 */
{
    int i;
    testdo(facbegsys, fsys, 24); /* facbegsys-开始符号集合 */
    while(inset(sym, facbegsys)) /* 循环直到不是因子开始符号 */
    {
        if(...)
        ...
        testdo(fsys, facbegsys, 23); /* 跳过因子后的非法符号 */
    }
    return 0;
}
```

◇ 后跟符号集随调用深度逐层增加
增加的符号与调用位置相关

例：调用 `expression(fsys, ...)`;

在 `write` 语句的下一层，则为

`expression([rparen,comma]+fsys, ...)`;

在 `factor` 的下一层，则为

`expression([rparen]+fsys, ...)`;

附： `write` 语句的语法 `write(<exp>{,<exp>})`;

`factor` 的语法： `factor ::= ... | '(' exp ')'`

◇ 目标代码生成函数

```
#define gendo(a, b, c) if (-1 == gen(a, b, c)) return -1
.....

int gen(enum fct x, int y, int z )
{
    if (cx >= cxmax)
    {
        printf("Program too long");
        return -1;
    }
    code[cx].f = x;
    code[cx].l = y;
    code[cx].a = z;
    cx++;
    return 0;
}
```


✧ 例

```
int statement(bool* fsys, int* ptx, int lev)
{
    ...
    if (sym == ident)    /* 准备按照赋值语句处理 */
    {
        i = position(id, *ptx);
        ...
        gendo(sto, lev-table[i].level, table[i].adr);
    }
    ...
}
```

◇ 跳转指令与地址返填

```
if (sym == ifsym)          /* 准备按照 if c then s 语句处理 */
{
    getsymdo;
    conditiondo(...);      /* 调用条件处理（逻辑运算）函数*/
    if (sym == thensym)
        getsymdo;
    else
        error(16);         /* 缺少 then */
    cx1 = cx;               /* 保存当前指令地址 */
    gendo(jpc, 0, 0);      /* 生成条件跳转指令，跳转地址未知，暂时写0 */
    statementdo(fsys, ptx, lev); /* 处理 then 后的语句 */
    code[cx1].a = cx;      /* 地址返填 */
}
```

PL/0 编译程序的类P-code代码生成



清华大学

《编译原理》

(0) jmp 0 8	转向主程序入口
(1) jmp 0 2	转向过程p入口
(2) int 0 3	过程p入口,为过程p开辟空间
(3) lod 1 3	取变量b的值到栈顶
(4) lit 0 10	取常数10到栈顶
(5) opr 0 2	次栈顶与栈顶相加
(6) sto 1 4	栈顶值送变量c中
(7) opr 0 0	退栈并返回调用点(16)
(8) int 0 5	主程序入口开辟5个栈空间
(9) opr 0 16	从命令行读入值置于栈顶
(10) sto 0 3	将栈顶值存入变量b中
(11) lod 0 3	将变量b的值取至栈顶
(12) lit 0 0	将常数值0进栈
(13) opr 0 9	次栈顶与栈顶是否不等
(14) jpc 0 24	相等时转(24) (条件不满足转)
(15) cal 0 2	调用过程p
(16) lit 0 2	常数值2进栈
(17) lod 0 4	将变量c的值取至栈顶
(18) opr 0 4	次栈顶与栈顶相乘(2*c)
(19) opr 0 14	栈顶值输出至屏幕
(20) opr 0 15	换行
(21) opr 0 16	从命令行读取值到栈顶
(22) sto 0 3	栈顶值送变量b中
(23) jmp 0 11	无条件转到循环入口(11)
(24) opr 0 0	结束退栈

◇ 例

```
const a=10;
var   b,c;
procedure p;
begin
    c:=b+a;
end;
begin
    read(b);
    while b#0 do
    begin
        call p;
        write(2*c);
        read(b);
    end
end.
```

(1) 仔细阅读类P-code解释程序

```
void interpret( )
```

初步理解相关的子程序

```
int base(...)
```

(2*) 仔细阅读PL/O编译程序的词法分析子程序

```
int getsym( )
```

及其相关的子程序

```
int getch( )
```

- (3*) 读懂各语法单位相应子程序的结构以及他们之间的调用关系
- (4*) 读懂各语法单位相应子程序内部的语义处理部分，理解符号表栈的结构及（随处理进程的）变化情况
- (5*) 读懂各语法单位相应子程序内部的错误处理部分的代码
- (6*) 读懂各语法单位相应子程序内部的代码生成部分的代码（特别是地址反填技术）

Thank You